

Day 17

1. Employee Salary Analysis Problem

A company stores employees' monthly salaries. Find:

- Highest salary • Lowest salary • Average salary • Number of employees earning above average

Input Enter salaries: 50000 60000 45000 80000 70000 55000

Expected Output

Highest Salary : 80000 Lowest Salary : 45000 Average Salary : 60000.0

Employees earning above average : 2

```
[ ] 1 import numpy as np
    2 n = 6
    3 sal = []
    4 print("Enter the input salaries:")
    5 for i in range(n):
    6     m = int(input(f"Enter salary {i+1}: "))
    7     sal.append(m)
    8 salary = np.array(sal)
    9 print(np.max(salary))
   10 print(np.min(salary))
   11 print(np.average(salary))
   12 print(np.sum(salary > np.mean(salary)))
```

```
... Enter the input salaries:
Enter salary 1: 50000
```

riables Terminal

Employees earning above average : 2

```
[ ] 1 import numpy as np
    2 n = 6
    3 sal = []
    4 print("Enter the input salaries:")
    5 for i in range(n):
    6     m = int(input(f"Enter salary {i+1}: "))
    7     sal.append(m)
    8 salary = np.array(sal)
    9 print(np.max(salary))
   10 print(np.min(salary))
   11 print(np.average(salary))
   12 print(np.sum(salary > np.mean(salary)))
```

```
... Enter the input salaries:
Enter salary 1: 50000
Enter salary 2: 60000
Enter salary 3: 45000
Enter salary 4: 80000
Enter salary 5: 70000
Enter salary 6: 55000
80000
45000
60000.0
2
```

2. Online Store Sales Analysis Problem An e-commerce website records daily sales for a week.

Find:

• Total sales • Average sales • Best sales day • Worst sales day

Input 25000 30000 18000 40000 50000 35000 28000

Expected Output

Total Sales : 226000

Average Sales : 32285.71

Best Day Sales : 50000

Worst Day Sales : 18000

```
[ ] import numpy as np
arr = np.array([25000, 30000, 18000, 40000, 50000, 35000, 28000])
print(np.sum(arr))
print(np.mean(arr))
print(np.max(arr))
print(np.min(arr))
```

50000
32285.714285714286
50000
18000

3. Student Marks Processing Problem A teacher stores marks of students.

Find: • Pass count (≥ 35) • Fail count • Topper mark • Average mark

Input 90 45 32 67 89 23 76 55 Expected Output Passed Students : 6 Failed Students : 2 Highest Mark : 90 Average Mark : 59.63

```
[ ] import numpy as np
arr = np.array([90, 45, 32, 67, 89, 23, 76, 55])
print(np.sum(arr>=35))
print(np.sum(arr<=35))
print(np.max(arr))
print(np.average(arr))
```

6
2
90
59.625

4. Warehouse Stock Monitoring Problem A warehouse stores product quantities. Find:

• Total stock • Products with stock less than 20 • Highest stock

Input 12 45 78 5 90 18 35 Expected Output Total Stock : 283 Low Stock Products : [12 5 18] Highest Stock : 90

```
[ ] import numpy as np
arr = np.array([12, 45, 78, 5, 90, 18, 35])
print(np.max(arr))
print(np.sum(arr))
print(arr[arr < 20])
```

90
283
[12 5 18]

5. Hospital Patient Temperature Analysis Problem Hospital stores patients' body temperatures. Find: • Average temperature

• Number of fever patients ($>37.5^{\circ}\text{C}$)

• Highest temperature

Input

36.5 37.8 38.2 36.9 39.1 37.0 Expected Output Average Temperature : 37.58 Fever Patients : 3 Highest Temperature : 39.1

```
[ ] import numpy as np
arr = np.array([36.5, 37.8, 38.2, 36.9, 39.1, 37.0])
print(np.average(arr))
print(np.sum(arr>37.5))
print(np.max(arr))
```

37.583333333333336
3
39.1

6. Bank Transaction Summary Problem

Positive numbers represent deposits and negative numbers represent withdrawals. Find:

- Total deposits • Total withdrawals • Final balance

Input 5000 -1000 3000 -500 2000 -1500

Expected Output Total Deposits : 10000

Total Withdrawals : 3000

Final Balance : 7000

```
import numpy as np
arr = np.array([5000, -1000, 3000, -500, 2000, -1500])
print(np.sum(arr[arr > 0]))
print(np.sum(arr[arr < 0]))
print(np.sum(arr))
```

```
... 10000
    -3000
    7000
```

7. Mobile App User Ratings Problem Store user ratings. Find:

- Average rating
- Number of 5-star ratings
- Lowest rating

Input

5 4 5 3 2 5 4 5

Expected Output Average Rating : 4.125 5-Star Ratings : 4 Lowest Rating : 2

```
import numpy as np
arr = np.array([5, 4, 5, 3, 2, 5, 4, 5])
print(np.average(arr))
print(np.sum(arr==5))
print(np.min(arr))
```

```
... 4.125
    4
    2
```

8. Electricity Consumption Analysis Problem Store daily electricity usage (units).

Find: • Total units • Average units • Days where usage exceeded 250 units Input 200 180 260 300 240 275 220 Expected Output Total

Units : 1675 Average Units : 239.29 High Usage Days : [260 300 275]

```
import numpy as np
arr = np.array([200, 180, 260, 300, 240, 275, 220])
print(np.sum(arr))
print(np.average(arr))
print(np.sum(arr > 250))
```

```
1675
239.28571428571428
[260 300 275]
None
```

9. Delivery Time Analysis Problem A logistics company stores delivery times (minutes). Find: • Fastest delivery

- Slowest delivery
- Average delivery time
- Deliveries completed within 30 minutes

Input 25 40 35 28 20 45 32

Expected Output

Fastest Delivery : 20 minutes Slowest Delivery : 45 minutes Average Delivery Time : 32.14 minutes Deliveries within 30 minutes : 3

```
import numpy as np
arr = np.array([25, 40, 35, 28, 20, 45, 32])
print(np.min(arr))
print(np.max(arr))
print(np.average(arr))
print(np.sum(arr <= 30))
```

```
... 20
    45
    32.142857142857146
    3
```

10. Social Media Post Engagement Problem Store likes received by different posts. Find:

- Total likes
- Most liked post
- Least liked post
- Posts with more than 500 likes

Input

120 450 670 890 320 540 210

Expected Output Total Likes : 3200 Most Likes : 890 Least Likes : 120 Popular Posts : [670 890 540]

```
import numpy as np
arr = np.array([120, 450, 670, 890, 320, 540, 210])
print(np.sum(arr))
print(np.max(arr))
print(np.min(arr))
print(arr[arr >= 540])
```

```
... 3200
    890
    120
    [670 890 540]
```